

Cahier des Charges : Résolution du Problème NP-Difficile de Kemeny.

Goales Saphiya

Teibi Zahra

Introduction

La théorie du vote s'intéresse aux règles qui permettent de trouver une décision collective à partir des préférences individuelles. Parmi les différentes méthodes existantes, l'agrégation de classement constitue un problème fondamental au cœur de la théorie du choix social, avec des applications allant de l'apprentissage automatique à la recherche web [1].

Parmi les règles de vote, la règle de Kemeny établit un classement de consensus qui minimise la distance totale de Kendall-tau (le nombre total de désaccords (paires inversées) entre le consensus et le profil de préférences). Le profil de préférences est l'ensemble $P = \{P_1, P_2, \dots, P_n\}$, où chaque P_i est le classement donné par le votant i .

D'un point de vue théorique, la règle de Kemeny possède des propriétés robustes, telles que le fait de satisfaire à la fois :

- le critère de Condorcet, c'est à dire que si un candidat, confronté à chaque autre candidat, est toujours gagnant (vainqueur de Condorcet), alors il sera classé premier dans le classement de Kemeny [2, 3].
- l'axiome de renforcement, qui stipule que si l'on considère deux groupes distincts de votants ayant les mêmes résultats, alors ce même résultat doit être obtenu lorsque l'on combine les votes des deux groupes [3].
- la neutralité qui garantit que si deux candidats échangent leurs positions dans les préférences de tous les votants, ils doivent également échanger leurs positions dans le classement final. Ainsi, tous les candidats sont traités de manière égale [3].

Problématique : cette solidité théorique est confrontée à une difficulté pratique majeure : sa complexité. En effet, le calcul d'un classement de Kemeny optimal est un problème NP-difficile [4, 5]. Ce résultat est valable même lorsque le nombre de votants est petit, pour $n = 4$, par exemple [5].

Afin de contrer cette problématique, des méthodes de décompositions ont été proposées. Ces méthodes réduisent l'instance originale à de plus petits sous-problèmes indépendants qu'il faudra ensuite résoudre.

Ce projet vise à implémenter des approches permettant de résoudre le problème de Kemeny de manière exacte et efficace, même sur des instances de grande taille.

Contexte et objectifs

L'objectif est alors de comparer les méthodes de résolution (programmation dynamique et programmation linéaire) ainsi que les méthodes de décomposition (le critère de Condorcet étendu [6] et la règle de majorité des 3/4 [4]). Afin d'obtenir des résultats d'évaluation réalistes, nous évaluerons ces méthodes sur des instances réelles stockées dans la bibliothèque Preflib. De plus, ce projet se concentre principalement sur des préférences complètes et sans égalité.

La réduction de données vise à identifier et à fixer des relations d'ordre entre candidats qui sont garanties d'apparaître dans toute solution optimale. Cette réduction constitue une phase de prétraitement essentielle pour rendre la résolution d'instances de grande taille abordable.

La règle de majorité des 3/4 [4]

La règle de majorité des 3/4 repose sur la notion de préférence forte et permet d'identifier des candidats dont la position relative peut être déterminée sans ambiguïté.

Pour tout candidat a et b , on note $a \succ_{3/4} b$, si a est préféré à b par au moins 3/4 des votants. Un candidat a est propre si et seulement si pour tout autre candidat b on a $a \succ_{3/4} b$ ou $b \succ_{3/4} a$.

Les candidats non propres nécessitent un traitement plus approfondi, car leur position relative reste incertaine même après application de la règle de majorité des 3/4. Le classement final de Kemeny s'obtient en concaténant les classements résolus des sous-instances et des candidats propres.

Exemple 1. Soit $C = \{a, b, c, d\}$ un ensemble de candidats et P un profil de préférences tels que $a \succ_{3/4} b$, $a \succ_{3/4} c$, $a \succ_{3/4} d$ et $c \succ_{3/4} d$. Il n'existe donc pas de préférence forte entre b et c . Ainsi, b et c ne sont pas propres. De même, puisque pour tout autre candidat (b, c, d), il existe une préférence forte impliquant a , alors a est un candidat propre.

Le critère de Condorcet étendu [6]

Le critère de Condorcet étendu constitue une version plus robuste du critère de Condorcet, qui se concentre sur un unique vainqueur.

Le principe est le suivant : si un sous-ensemble de candidat C_α est majoritairement préféré à tous les candidats de l'ensemble complémentaire $C_\beta = C \setminus C_\alpha$ alors pour tout $x \in C_\alpha$ et $y \in C_\beta$, x est préféré à y par plus de la moitié des électeurs. Ainsi cet ensemble C_α doit se placer en tête de tout classement optima.

Ce principe peut être généralisé avec plusieurs sous-ensembles formant une partition de l'ensemble des candidats C .

Le classement de Kemeny sur C est obtenu en juxtaposant les classements de Kemeny trouvés sur les sous-ensembles d'une partition.

Algorithme du critère de Condorcet étendu :

1. construction du graphe de majorité stricte
2. ordonnancement des composantes fortement connexes

Programmation Dynamique [7]

L'algorithme de programmation dynamique calcule le score de Kemeny (la somme des distances de Kendall-Tau) pour chaque sous-ensemble possible de candidats $C' \subseteq C$ et renvoie la solution qui minimise le score total.

Décomposition récursive :

Cas de base : pour les sous-ensembles C' de taille 1, le calcul d'un classement de Kemeny est trivial. Il y a un seul candidat c et le score de Kemeny associé est de 0.

Étape récurrente : pour les sous-ensembles C' de taille $|C'| > 1$, l'algorithme teste itérativement chaque candidat $c \in C'$ comme étant celui placé en première position du classement local.

Calcul du score : le score total pour c est : $\text{score}(c, C') = \text{cout}(c, C') + \text{score}(C'')$. Le $\text{score}(C'')$ est le score optimal déjà calculé pour l'ensemble des candidats restants C'' et le $\text{cout}(c, C')$ est le coût engendré par le placement de c en tête du classement C' (nombre d'insatisfactions).

Optimisation : le candidat c qui minimise le score total, est donc placé en première position dans le classement optimal pour ce sous-ensemble C' .

Programmation linéaire [4]

L'agrégation de classement de Kemeny nécessite que le classement final soit un ordre strict. Ce problème peut donc être modélisé par le programme linéaire en nombres entiers suivant :

$$\min \sum_{a,b} Q_{ab} \cdot x_{ba} + Q_{ba} \cdot x_{ab} \quad (1)$$

$$x_{ab} + x_{ba} = 1 \quad (2)$$

$$x_{ab} + x_{bc} + x_{ca} \geq 1 \quad (3)$$

$$x_{ab} \in \{0, 1\} \quad (4)$$

Ce programme linéaire vise à minimiser le score total de Kemeny et retourne une solution du problème d'agrégation de Kemeny.

Le modèle utilise les variables binaires $x_{ab} \in \{0, 1\}$ qui valent 1 si le candidat a est classé avant le candidat b dans le classement de Kemeny et les variables Q_{ab} représentant le nombre de fois où le candidat a est classé avant b dans le profil de préférences.

Les contraintes suivantes garantissent que les solutions x_{ab} représentent un ordre linéaire valide :

- Contraintes d'ordre strict (contrainte (2)) : cette contrainte assure que pour toute paire de candidats (a, b) , on ne peut pas avoir à la fois $a \succ b$ et $b \succ a$ dans le classement final.
- Contraintes de transitivité (contrainte (3)) : cette contrainte est essentielle car elle évite l'existence de cycle dans le classement.

Implémentation

Nous allons développer ce projet en python.

Nous devons définir une classe *Instance* afin d'y stocker l'instance sur laquelle nous évaluons nos méthodes. Celle-ci sera composée des attributs et méthodes suivants :

attributs

- nb_votants : correspondant au nombre de votants
- nb_candidats : correspondant au nombre de candidats
- matPref : une matrice qui compte les préférences. $\text{matPref}[i, j]$ correspond au nombre de votants qui préfèrent i à j .
- candidats : un dictionnaire associant le numéro de chaque candidat à son nom

methodes

- lecture_fichier : initialise l'*Instance* à partir d'un fichier .soc contenant une instance complète et sans égalité (les données réelles de Preflib)
- construction_graphe_majorite : construit une instance de la classe *Graphe*, ajoute un nœud par candidat et un arc entre deux candidats a et b si a est préféré par plus de 50% des votants à b .

La classe *Graphe* contiendra les attributs et méthodes suivantes :

attributs

- noeuds : ensemble de noeuds
- voisins : un dictionnaire d'adjacence

methodes

- `__str__` : une méthode d’affichage du graphe sur le terminal
- `afficher_graphe` : une méthode d’affichage du graphe plus visuel avec matplotlib
- `getNoeuds` : retourne l’ensemble des noeuds
- `addNoeud` : ajoute un noeud au graphe
- `addVoisins` : ajoute un voisin à un noeud
- `getVoisins` : retourne les voisins d’un noeud

Pour procéder à la décomposition de l’instance, on aura également besoin des méthodes *majorite_trois_quart* et *critere_condorcet_etendu* (à l’aide de l’algorithme de Tarjan). De même, pour les méthodes de résolutions exactes, nous aurons besoin des méthodes *resolution_PD* et *resolution_PL*.

Pour finir, nous implémenterons des méthodes d’évaluation telles que *evaluation_temps_calcul* qui estimera le temps d’exécution des différentes méthodes de résolutions (avec et sans décomposition). Nous aurons également une méthode *evaluation_decomposition* qui estimera la capacité des méthodes à décomposer l’instance en sous-instances, c’est-à-dire la taille de la plus grande sous-instance obtenue après la réduction.

Un des enjeux de l’implémentation est d’avoir des algorithmes de résolutions applicables aux sous-instances et des algorithmes de décomposition qui génèrent des sous-instances. Ceci soulève beaucoup de questions sur l’implémentation telles que :

- comment gérer les sous-instances ?
- comment allier les méthodes de décompositions ?
 - règle de majorité des $3/4$ puis Critère de condorcet étendu
 - ou Critère de condorcet étendu puis règle de majorité des $3/4$
 - ou règle de majorité des $3/4$
 - ou Critère de condorcet étendu
- est-il possible de paralléliser les résolutions sur les sous-instances ?

Références

- [1] Dwork Cynthia, Kumar Ravi, Moni Naor, and D. Sivakumar. Rank aggregation methods for the web. 2001.
- [2] Davenport Andrew and Kalagnanam Jayant. A computational study of the kemeny rule for preference aggregation. 2004.
- [3] Young H. P. and Levenglick A. A consistent extension of condorcet’s election principle. 1978.
- [4] Nadja Betzler, Robert Bredereck, and Rolf Niedermeier. Theoretical and empirical evaluation of data reduction for exact kemeny rank aggregation. 2013.
- [5] Fischer Felix, Hudry Olivier, and Niedermeier Rolf. Weighted tournament solutions. 2016.
- [6] Truchon Michel. An extension of the condorcet criterion and kemeny orders. Technical report, 1999.
- [7] Nadja Betzler, Michael R. Fellows, Jiong Guo, Rolf Niedermeier, and Frances A. Rosamond. Fixed-parameter algorithms for kemeny rankings. 2009.